操作系统

概念

- 负责协调软硬件等计算机资源的工作，为上层用户、应用程序提供简单易用的服务的软件系统

功能

- 资源的管理者
  1. 处理机管理 —— 进程控制/进程同步/进程通信/进程调度
  2. 存储器管理 —— 内存分配/内存保护/地址映射/内存扩充
  3. 文件管理 —— 文件存储空间管理/目录管理/文件读写管理/文件保护
  4. 设备管理 —— 缓冲管理/设备分配/设备处理
- 向用户提供服务
  - **用户接口**
    - 允许用户直接使用
    - **联机命令接口**　用户说一句，系统说一句（如终端输入命令）
    - **脱机命令接口**　用户说一句，系统说一堆（如.bat文件批处理）
    - **图形用户接口**
  - **程序接口**
    - 用户通过程序间接使用
    - 由一组系统调用组成
- 对硬件机器的扩展　扩充机器

特征

1. **并发** —— 宏观上多个程序同时运行，微观上单处理机中程序分时交替进行
2. **共享**
   - 互斥共享方式　对摄像头设备的共享使用
   - **同时共享方式**　**对硬盘资源的共享使用**
3. **虚拟**
   - 通过某种技术将一个物理实体变成若干个逻辑上的对应物
   - **空分复用技术**
     - 利用存储器的空闲空间分区域存放和运行其他的多道程序
     - 虚拟存储技术
   - **时分复用技术**
     - 利用处理机的空闲时间运行其他程序
     - 虚拟处理机技术、虚拟设备技术
4. **异步**　进程以不可预知的速度向前推进

发展与分类

1. **手工操作阶段**　缺点：人机速度矛盾
2. **单道批处理系统**
   - 引入脱机输入输出技术，监督程序控制流程
   - 优点：缓解人机速度矛盾
   - 缺点：资源利用率低，cpu与I/O串行操作
3. **多道批处理系统**
   - 操作系统开始出现
   - 多道程序并发执行，资源利用率高
   - 平均周转时间长，不提供人机交互功能
4. **分时操作系统**
   - 优点：提供人机交互功能
   - 缺点：不能优先处理紧急任务
5. **实时操作系统**
   - **硬实时操作系统**　**必须在严格的规定时间内完成处理**
   - **软实时操作系统**　**能接收偶尔违反时间规定**
6. **网络操作系统、分布式操作系统、个人计算机操作系统**

操作系统

运行机制

- 两种指令
  - 非特权指令 —— 如运算指令
  - 特权指令 —— 如I/O指令，内存清零指令
- 两种处理器状态
  - 用户态
  - 核心态
- 两种程序
  - 内核程序
  - 应用程序

体系结构

系统内核

- 对系统进行管理的功能
  1. 进程管理
  2. 存储器管理
  3. 设备管理
- **时钟管理** —— 实现计时功能
- **中断处理** —— 负责实现中断机制
- **原语**
  1. 特殊的程序，初一操作系统的最底层，最接近硬件
  2. 具有原子性，运行只能一气呵成不能终端
  3. 运行时间较短，调用频繁

大内核

- 将操作系统的主要功能模块都作为系统内核
- 优点：高性能；
- 缺点：内核代码庞大，结构混乱，难以维护

微内核

- **基本概念**
  - **足够小的内核**
    1. 与硬件处理紧密相关的部分
    2. 一些较基本的功能
    3. 客户与服务器之间的通信
  - **基于客户/服务器模式**
    - 将操作系统的绝大部分功能放在微内核外面的一组服务器（进程）中实现
    - 如进程/线程管理服务器、虚拟存储服务器、I/O设备管理服务器
    - 运行在用户态、客户与服务器之间借助微内核提供的消息传递机制实现消息交互
  - **应用“机制与策略分离”原理**
    - 机制是指实现某一功能的具体执行机构，处于系统底层
    - 策略是在机制的基础上借助某些参数和算法实现功能优化，处于系统高层
  - **采用面向对象技术**
    - “抽象”和“屏蔽”原则控制系统的复杂性
    - “对象”“封装”“继承”确保操作系统的正确性、可靠性、易扩展性、易修改性
- **基本功能**
  - 进程/线程管理
    - 将指定优先级进程/线程从所在队列中取出投入运行，属于调度功能的机制部分，放入微内核中
    - 对用户进程/线程分类及优先级确认的方式或原则属于策略问题，放入微内核的进程/线程管理服务器中
  - 低级存储器管理
    - 配置最基本的低级存储器管理机制
    - 如用于实现将用户空间的逻辑地址变换为内存空间的物理地址的页表机制和地址变换机制
  - 中断和陷入处理
    - 捕获所发生的中断和陷入事件，进行现场保护，识别中断和陷入的类型
    - 将有关事件转换成消息后发送给相关的服务器，调用相应的处理程序来进行后期处理
- 优点 —— 提高了系统的可扩展性，增强了系统的可靠性，可移植性强，提供了对分布式系统的支持，融入了面向对象技术
- 问题 —— 客户和服务器、服务器和服务器之间的通信都需经过微内核，上下文切换频繁

系统调用

- 作用
  - 系统调用会使处理器从用户态进入核心态
  - 应用程序通过系统调用来请求获得操作系统的服务
- 分类
  - 文件管理　　完成文件的 读/写/创建/删除 等功能
  - 设备管理　　完成设备的 请求/释放/启动 等功能
  - 进程控制　　完成进程的 创建/撤销/阻塞/唤醒 等功能进
  - 进程通信　　完成进程之间的 消息传递/信号传递 等功能
  - 内存管理　　完成内存的 分配/回收 等功能
- 系统调用与库函数的区别
  - 统调用是操作系统向上层提供的接口
  - 有的库函数是对系统调用的进一步封装
  - 当今编写的应用程序大多是通过高级语言编写的库函数进行系统调
- 过程
  1. 传递系统调用参数
  2. 执行陷入指令
  3. 进行系统调用相应服务进程
  4. 返回用户程序

中断和异常
中断机制的诞生
为了实现多道程序并发执行而引入的一种技术
中断的概念和作用
发生中断, 就意味着需要操作系统介入中断的概念和作用 开展管理工作, cpu会立即进入核心态
中断是cpu进入核心态的唯一途径
中断的分类
外中断 (中断)
外设请求
I/O操作完成发出的中断信号
人工干预
用户强行终止一个进程
内中断 (异常/例外/陷入)
强迫中断
硬件故障
软件故障
自愿中断 —— 指令中断
补充: 内中断的另一种方式
陷阱/陷入
有意为之的异常, 如系统调用
故障
由错误条件引起可能被故障处理程序修复, 如缺页
终止
不可恢复的致命错误造成的结果, 终止程序不再将控制返回给引发终止的应用程序, 如整数除0
外中断的处理过程
1
每条指令执行结束后, cpu检查是否有外部中断信号
2
若有外部中断信号, 则需要保护中断进程的cpu环境
3
根据中断信号类型转入相应的中断处理程序
4
恢复原进程的cpu环境并退出中断, 返回原进程执行

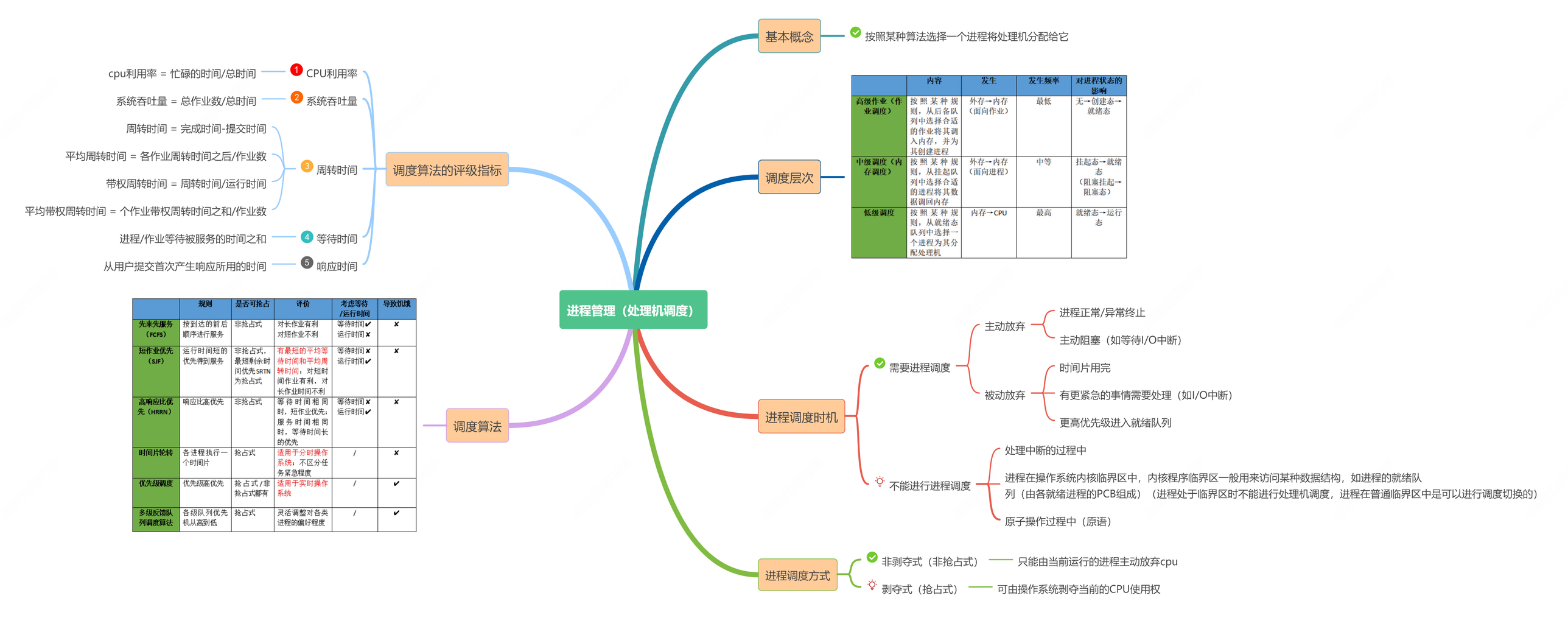
进程管理（进程）

- 定义
  - 进程实体的运行过程，是系统进行资源分配和调度的一个独立单位
- 特征
  1. **动态性** —— 进程是程序的一**次执行过程**，是动态地产生、变化和消亡的
  2. **并发性** —— 内存中有多个进程实体，各进程可并发执行
  3. **独立性** —— **独立运行/获得资源/接受调度**的基本单位
  4. **异步性** —— 各进程按各自**独立的不可预知的速度**向前推进，可能**导致运行结果的不确定性**
  5. **结构性** —— 每个进程都配置一个PCB对其进行描述，结构上看由程序段、数据段和PCB组成
- 进程的组成
  - **PCB进程控制块**
    - 进程描述信息 —— 进程标识符PID/用户标识符UID
    - 进程控制和管理信息 —— 进程当前状态/优先级
    - 资源分配清单 —— 代码段/数据段/堆栈段指针 文件描述符 键盘/鼠标
    - 处理机相关信息 —— 通用/地址/控制/标志寄存器值 状态字
  - **程序段** —— 存放要执行的代码
  - **数据段** —— 存放程序运行过程中处理的各种资源
- 进程的组织方式
  - **链接方式**
    - 按照进程状态将PCB分为多个队列
    - 操作系统持有指向各个队列的指针
  - **索引方式**
    - 按照进程状态建立几张索引表
    - 操作系统持有指向各索引表的指针
- 进程的状态与转换
  - 状态
    - 运行态 —— cpu ✓ 其他所需资源✓
    - 就绪态 —— cpu ✗ 其他所需资源✓
    - 阻塞态/等待态 —— cpu ✗ 其他所需资源✓
    - 创建态 —— 操作系统为新进程分配资源、创建PCB
    - 结束态 —— 操作系统回收进程资源、撤销PCB
  - 状态转换
    1. 就绪态→运行态 —— 进程被调度
    2. 运行态→就绪态 —— 时间片到/cpu被高优先级进程抢占
    3. 运行态→阻塞态 —— 等待系统资源分配/等待某事件发生 (主动行为)
    4. 阻塞态→就绪态 —— 资源分配到位/等待的事件发生 (被动行为)
    5. 创建态→就绪态 —— 系统完成创建进程相关工作
    6. 运行态→终止态 —— 进程运行结束/运行过程中遇到不可修复的错误
- 进程控制
  - 进程的创建/终止/阻塞/唤醒/切换
- 进程的通信
  1. 共享存储
     - 互斥地访问共享空间
     - **①基于数据结构的共享 (低级通信)** —— 如共享空间中放置长度为10的数组，速度慢
     - **②基于存储区的共享 (高级通信)** —— 进程控制数据形式/存放位置，速度快
  2. 消息传递
     - 传递结构化消息 (消息头/消息体)，系统提供"发送/接受原语"
     - **①直接通信方式** —— 消息直接挂到接收方的消息队列里
     - **②间接通信方式** —— 消息先发送到中间体 (信箱)
  3. 管道通信
     - 各进程互斥地访问管道
     - 设置一个特殊的共享文件 (管道)，也就是缓冲区只
     - 能实现半双工通信，实现双向通信要建立两个管道写满
     - 时不能再写，读空时不能再读
     - 没写满不能读，没读空不能写

进程管理（线程）

- 定义
  - "轻量级进程"
- 变化
  - 资源分配/调度
    - 传统机制中, 进程是资源分配/调度的基本单位
    - 引入线程后, 进程是资源分配的基本单位, 线程是处理机调度的基本单位
  - 并发性
    - 传统机制中, 只有进程间并发
    - 引入线程后, 各线程间也并发, 提高了并发度
  - 系统开销
    - 传统的进程并发, 需要切换进程的运行环境, 系统开销很大
    - 线程间并发, 同一进程的线程切换不会引起进程切换, 系统开销小
- 属性
  - 线程几乎不拥有系统资源
  - 多cpu计算机中, 各个线程可以占用不同的cpu
  - 每个线程都有一个线程ID、线程控制块 (TCB)
  - 线程也有就绪/阻塞/运行三种基本状态
  - 同一进程中的不同线程间共享进程的资源
- 线程的实现方式
  - 用户级线程 —— 从用户视角看的线程
  - 系统级线程 —— 从操作系统角度看的线程 (内核级线程才是处理机分配的单位)
- 多线程模型

	优点	缺点
多对一模型	线程管理开销小效率高	一个线程阻塞会导致整个进程都被阻塞 (并发度低)
一对一模型	各个线程可分配到多核处理机 并行执行 (并发度高)	线程管理开销大
多对多模型	集二者之长	


进程管理（处理机调度）
基本概念
按照某种算法选择一个进程将处理机分配给它
调度层次
内容
发生
发生频率
对进程状态的影响
高级作业（作业调度）
按照某种规则，从后备队列中选择合适的作业将其调入内存，并为其创建进程
外存→内存（面向作业）
最低
无→创建态→就绪态
中级调度（内存调度）
按照某种规则，从挂起队列中选择合适的进程将其数据调回内存
外存→内存（面向进程）
中等
挂起态→就绪态（阻塞挂起→阻塞态）
低级调度
按照某种规则，从就绪态队列中选择一个进程为其分配处理机
内存→CPU
最高
就绪态→运行态
进程调度时机
需要进程调度
主动放弃
进程正常/异常终止
主动阻塞（如等待I/O中断）
被动放弃
时间片用完
有更紧急的事情需要处理（如I/O中断）
更高优先级进入就绪队列
不能进行进程调度
处理中断的过程中
进程在操作系统内核临界区中，内核程序临界区一般用来访问某种数据结构，如进程的就绪队列（由各就绪进程的PCB组成）（进程处于临界区时不能进行处理机调度，进程在普通临界区中是可以进行调度切换的）
原子操作过程中（原语）
进程调度方式
非剥夺式（非抢占式）
只能由当前运行的进程主动放弃cpu
剥夺式（抢占式）
可由操作系统剥夺当前的CPU使用权
调度算法
规则
是否可抢占
评价
考虑等待/运行时间
导致饥饿
先来先服务（FCFS）
按到达的前后顺序进行服务
非抢占式
对长作业有利 对短作业不利
等待时间✔ 运行时间✘
✘
短作业优先（SJF）
运行时间短的优先得到服务
非抢占式，最短剩余时间优先SRTN为抢占式
有最短的平均等待时间和平均周转时间；对短时间作业有利，对长作业时间不利
等待时间✘ 运行时间✔
✘
高响应比优先（HRRN）
响应比高优先
非抢占式
等待时间相同时，短作业优先；服务时间相同时，等待时间长的优先
等待时间✘ 运行时间✔
✘
时间片轮转
各进程执行一个时间片
抢占式
适用于分时操作系统；不区分任务紧急程度
/
✘
优先级调度
优先级高优先
抢占式/非抢占式都有
适用于实时操作系统
/
✔
多级反馈队列调度算法
各级队列优先机从高到低
抢占式
灵活调整对各类进程的偏好程度
/
✔
调度算法的评级指标
1 CPU利用率
cpu利用率 = 忙碌的时间/总时间
2 系统吞吐量
系统吞吐量 = 总作业数/总时间
3 周转时间
周转时间 = 完成时间-提交时间
平均周转时间 = 各作业周转时间之后/作业数
带权周转时间 = 周转时间/运行时间
平均带权周转时间 = 个作业带权周转时间之和/作业数
4 等待时间
进程/作业等待被服务的时间之和
5 响应时间
从用户提交首次产生响应所用的时间

进程管理（进程同步）

制约关系

- 同步：并发性带来了异步性，需要在某些位置上协调进程之间的工作次序，遵循一定的先后顺序
- 互斥：对临界资源的访问，需要互斥的进行。同一时间段内只能允许一个进程访问该资源
  - 四个部分
    1. 进入区：检查是否可进入临界区，若可进入，则需"上锁"
    2. 临界区：访问临界资源的代码
    3. 退出区："解锁"
    4. 剩余区：其余代码部分

临界区互斥

- 原则
  1. **空闲让进**：临界区空闲时，应允许一个进程访问
  2. **忙则等待**：临界区正在被访问时，其他试图访问的进程需要等待
  3. **有限等待**：要在有限时间内进入临界区，保证不会饥饿
  4. **让权等待**：进不了临界区的进程，要释放处理机，防止忙等
- 基本方法

		算法思想	主要问题
软件	1. **单标志法**	进入区只"检查"不"上锁"，退出区把临界区的使用权转交给另一个进程，即每个进程进入临界区的权限只能被另一个进程赋予	违背"空闲让进"原则
	2. **双标志先检查**	进入区先"检查"后"上锁"，退出区"解锁"	违背"忙则等待"原则
	3. **双标志后检查**	先"上锁"后"检查"，退出区"解锁"	违背"空闲让进"和"有限等待"原则，可能导致饥饿
	4. **皮特森算法**	进入区"主动争取—主动谦让—检查对方是否想进—己方是否谦让	违背"让权等待"原则 发生"忙等"
硬件	5. **中断屏蔽方法**	使用"开/关中断"指令实现 优点：简单高效 缺点：只适用于单处理机，只适用于操作系统内核进程	
	6. **TestAndSet (TS指令/TSL指令)**	优点：实现简单；适用于多处理机环境 缺点：不满足"让权等待"	
	7. **swap指令**	(XCHG指令) 逻辑上同TSL	
信号量	8. **利用PV操作实现互斥**	以上7种方法都无法实现"让权等待"	

信号量机制

- 类型
  - 整形型信号量
    - 用一个整数型变量作为信号量，数值表示某种资源数
    - 整型信号量与普通整型变量的区别：对信号量只能执行初始化、P、V三种操作
    - 整型信号量存在的问题：不满足"让权等待"原则
  - 记录型信号量
    - S.value表示某种资源数，S.L指向该等待队列
    - P操作中，一定是先S.value-，之后可能需要执行block原语
    - V操作中，一定是先S.value++，之后可能需要执行wakeup原语
    - 实现系统资源的"申请"和"释放"
- 实现进程互斥
  1. 分析问题，确定临界区
  2. **设置互斥信号量，初始值为1**
  3. 临界区之前对信号量执行P操作
  4. 临界区之后对信号量执行V操作
- 实现进程同步
  1. 分析问题，找出"一前一后"的同步关系
  2. **设置同步信号量，初始值为0**
  3. 在"前操作"之后执行V操作
  4. 在"后操作"之前执行P操作
- 实现进程的前驱关系
  1. 分析问题，画出前驱图，每一对前驱关系都看成一个同步问题
  2. 为每一对前驱关系设置同步信号量，初值为0
  3. 在每个"前操作"之后执行V操作
  4. 在每个"后操作"之前执行P操作
- PV操作题目的解题思路
  1. 关系分析。找出题目中描述的各个进程，分析它们之间的同步、互斥关系
  2. 整理思路。根据各进程的操作流程确定P、V操作的大致顺序
  3. 设置信号量。根据题目条件确定信号量初值（互斥信号量一般为1，同步信号量的初值对应资源的初值）

经典问题

- **生产者消费者问题**
- **多生产者-多消费者问题**
- **吸烟者问题**
- **读者-写者问题**
- **哲学家进餐问题**

管理

- 引入管程原因：解决信号量机制编程麻烦、易出错的问题
- 组成
  - 共享数据结构
  - 对数据结构初始化的语句
  - 一组用来访问数据结构的过程（函数）
- 基本特征
  - 各外部进程/线程只能通过管程提供的特定"入口"才能访问共享数据
  - 每次仅允许一个进程在管程内执行某个内部进程
- 补充
  - 各进程必须互斥访问管程的特性是由编译器实现的
  - 可在管程中设置条件变量及等待/唤醒操作以解决同步问题

进程管理（死锁）

产生原因

非剥夺资源的竞争和进程的不恰当推进顺序

概念区别

- **死锁**
  - 发生死锁的进程一定处于阻塞态
  - 至少有两个或两个以上的进程同时发生死锁
- **饥饿**
  - 进程可能是阻塞态 (长期得不到I/O设备) /就绪态 (长期得不到处理机)
  - 可能只有一个进程发生饥饿
- **死循环**
  - 由代码的逻辑错误导致
  - 可能只有一个进程发生死循环

必要条件

1. **互斥条件** — 争抢互斥使用的资源
2. **不剥夺条件** — 进程保持的资源只能主动释放不能强行剥夺
3. **请求和保持条件** — 保持某些资源不放的同时请求别的资源
4. **循环等待条件** — 存在进程资源循环等待链

处理策略

1 预防死锁 (静态策略) 不允许死锁发生

1. **破坏互斥条件**
   - 将互斥使用的资源改造为允许共享使用(如SPOOLing技术)
   - ⚠ 缺点: 可行性低, 很多时候无法破坏互斥条件
2. **破坏不剥夺条件**
   1. 申请的资源得不到满足时立即释放拥有的所有资源
   2. 申请的资源被占用时, 由操作系统协助剥夺(优先级)
   - ⚠ 缺点: 实现复杂: 系统开销大: 可能导致饥饿
3. **破坏请求和保持条件**
   - **采用静态分配法**, 运行前分配好所有资源, 之后一直保持
   - ⚠ 缺点: 资源利用率低: 可能导致饥饿
4. **破坏循环等待条件**
   - **采用顺序资源分配法**, 必须按编号从小到大的顺序申请资源
   - ⚠ 缺点: 不方便增加新设备: 导致资源浪费: 用户编程麻烦

2 避免死锁 (动态策略) 不允许死锁发生

银行家算法

- 数据结构
  - 最大需求矩阵: Max[]
  - 分配矩阵: Allocation[]
  - 需求矩阵: Need[]=Max-Allocation
  - 可用资源: Available[]
  - 申请资源: Request[]
- 算法步骤
  1. 检查Request是否超过Need
  2. 检查Available是否满足Request
  3. 试探分配, 更改数据结构
  4. 检查当前Available是否满足某个进程的Need, 可以则加入安全序列并把该进程所有资源回收
  5. ⑤重复上述步骤, 看最终能否让所有进程加入安全序列

3 检测死锁 允许死锁发生

资源分配图

- 数据结构
  - 进程结点/资源结点
  - 进程结点→资源结点: 请求边
  - 资源结点→进程结点: 分配边
- 算法步骤
  1. 找出既不阻塞又不是孤点的进程Pi, 消去所有请求边和分配边, 使之成为孤立的结点
  2. 重复上述步骤, 若能消去所有边, 则称该图是可完全简化的; 否则还连着边的进程就是死锁进程

4 解除死锁允许死锁发生

1. **资源剥夺法** — 挂起某些死锁进程并抢占资源分配给其他死锁进程
2. **撤销进程法** — 强制撤销部分甚至全部并剥夺资源
3. **进程回退法** — 让一个或多个进程回退到足以避免死锁的地步

内存管理

进程运行的基本原理

- 指令工作的原理 —— 操作码+若干参数 (可能包含地址参数)
- 逻辑地址 (相对地址) vs 物理地址 (绝对地址)
- 从写程序到程序运行
  1. **编辑** —— 源代码文件
  2. **编译** —— 由源代码文件生成目标模块 (高级语言"翻译"为机器语言)
  3. **链接** —— 由目标模块生成装入模块, 链接后形成完整的逻辑地址
     1. **静态链接** —— 装入前链接成一个完整模块
     2. **装入时动态链接** —— 运行前边装入边链接
     3. **运行时动态链接** —— 运行时需要目标模块时才装入并链接
  4. **装入** —— 将装入模块装入内存, 装入后形成物理地址
     1. **绝对装入** —— 编译时产生绝对地址 (单道程序阶段, 无操作系统)
     2. **静态重定位** —— 装入时将逻辑地址转换为物理地址 (早起多道批处理阶段)
     3. **动态重定位** —— 运行时将逻辑地址转换为物理地址, 需设置重定位寄存器 (现代操作系统)

内存空间的分配与回收

连续分配管理方式

1. 单一连接分配
   - 只支持单道程序, 内存分为系统区和用户区, 用户程序放在用户区
   - 无外部碎片, 有内部碎片
   - 可采用覆盖技术扩充内存, 不一定需要采取内存保护
2. 固定分区分配
   - 支持多道程序, 内存用户空间分为若干个固定大小的分区, 每个分区只能装一道作业无
   - 无外部碎片, 有内部碎片
   - 分区说明表: 分区号、大小、起始地址、状态
   - 两种分区方式
     - 分区大小相等
     - 分区大小不等
3. 动态分区分配
   - 支持多道程序, 在进程装入内存时, 根据进程的大小动态地建立分区
   - 无内部碎片, 有外部碎片 (由"紧凑"技术解决)
   - 两种数据结构
     - 空闲分区表: 分区号、分区大小、起始地址、状态
     - 空闲分区链
   - 回收内存分区
     - 回收区之后有相邻的空闲分区
     - 回收区之前有相邻的空闲分区
     - 回收区前后有相邻的空闲分区
     - 回收区前后无相邻的空闲分区

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找合适的分区	地址递增	综合性能最好, 算法开销小, 回收分区后一般无需对空闲分区队列重新排序	
最佳适应	优先使用更小的分区	容量递增	会有更多的大分区被保留下来, 满足大进程的需求	产生小的难以利用的碎片; 法开销大, 回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区	容量递减	减少难以利用的小碎片	大分区容易被用完,不利于大进程; 算法开大, 需对空区队列重
邻近适应	从上次查找结束位置开始	地址递增 (可排列成循环链表)	无需每次从低地址的小分区始检索; 算法开销小	会使高地址的大分区被用完

销
闲分
新排序

内存管理
基本分页存储管理
思想
将进程分页, 各个页面离散的放到各个内存块中
概念
"页框、页帧、内存块、物理块" vs "页、页面"
"页框号、页帧号、内存块号、物理块号" vs "页号、页面号"
页表
一个进程对应一张页表, 进程的每一页对应一个页表项
每个页表项由"页号"和"块号"组成
每个页表项的长度是相同的, 页号是"隐含"的
基本地址变换结构
页表寄存器
存放页表起始地址
存放页表长度
地址变换过程
1 根据逻辑地址算出逻辑页号、页内偏移量
2 页号的合法性检查 (与页表长度对比)
3 若页号合法, 再根据页表起始地址、页号找到对应页表项 (第一次访存: 查页表)
4 根据页表项中记录的内存块号、页内偏移量得到最终的物理地址
5 访问物理内对应的内存单元 (第二次访存: 访问目标内存单元)
几个细节
页式管理中地址是一维的
实际应用中, 通常使一个页框恰好能放入整数个页表项
为了方便找到页表项, 页表一般是放在连续的内存块中
具有快表的地址变换结构
局部性原理
时间局部性: 执行的某条指令可能再次被执行, 访问过的数据可能再次被访问
空间局部性: 访问的某个存储单元附近的存储单元可能被访问
快表（TLB）
联想寄存器, 访问速度比内存快很多的高速缓冲存储器, 存放当前访问的若干页表项
地址变换过程
1 根据逻辑地址算出逻辑页号、页内偏移量
2 页号的合法性检查 (与页表长度对比)
3 查快表, 若命中即可知道页面存放的内存块号, 可直接进行⑤, 未命中则进行④
4 查页表, 找到页面存放的内存块号, 并且将页表项复制到快表中
5 根据内存块号与页内偏移量得到物理地址
6 访问目标内存单元
快表命中只需一次访存, 未命中需两次访存
两级页表
单级页表的问题
所有页表必须连续存放, 页表过大时需要很大的连续空间
在一段时间内并非所有页面都用得到, 因此没必要让整个页表常驻内存将
两级页表
长的页表再分页
逻辑地址结构 (一级页号, 二级页号, 页内偏移量)
页目录表/外层页表/顶级页表
地址变换
1 按照地址结构将逻辑地址拆分成三部分
2 从PCB中读出页目录表始址, 根据一级页号查页目录表, 找到下一级页表在内存中的存放位置
3 根据二级页号查表, 找到最终想访问的内存块号
4 结合页内偏移量得到物理地址
几个细节
多级页表中, 各级页表大小不能超过一个页面。若两级页表不够, 可以分更多级
多级页表的访存次数 (假设没有快表机构) - N级页表访问一个逻辑地址需要N+1次访存
基本分段存储管理
分段
将地址空间按照程序自身的逻辑关系划分成若干个段, 每段从0开始编址
每个段在内存中占据连续空间, 但各段之间可以不相邻
逻辑地址结构: (段号, 段内地址)
段表
记录逻辑段到实际存储地址的映射关系
每个段表对应一个段表项, 各段表项长度相同, 由段号 (隐含) 、段长、基址组成
地址变换
1 由逻辑地址得到段号、段内地址
2 段号与段表寄存器中的段长度比较, 检查是否越界
3 由段表始址、段号找到对应段表项 (第一次访存)
4 根据段表中记录的段长, 检查段内地址是否越界
5 根据基址与段内偏移量得到物理地址
6 访问目标单元 (第二次访存)
分段VS分页
分页对用户不可见, 分段对用户可见
分页的地址空间是一维的, 分段的地址空间是二维的
分段更容易实现信息的共享和保护 (纯代码/可重入代码可以共享)
分页 (单级页表) 分段访问一个逻辑地址都需要两次访存, 分段存储中也可以引入快表机构将地址
段页式存储管理
分段+分页
空间按照程序自身的逻辑关系划分为若干个段, 再将各段分为大小相等的页面
将内存空间分为与页面大小相等的一个个内存块, 系统以块为单位为进程分配内存
逻辑地址结构: (段号, 页号, 页内偏移量)
段表、页表
每个段对应一个段表项。各段表项长度相同, 由段号 (隐含) 、页表长度、页表存放地址组成
每个页对应一个页表项。各页表项长度相同, 由页号 (隐含) 、页面存放的内存块号组成
地址变换
1 由逻辑地址得到段号、页号、页内偏移量
2 段号与段表寄存器中的段长度比较, 检查段号是否越界
3 由段表始址、段号找到对应段表项 (第一次访存: 查段表)
4 根据段表中记录的页表长度, 检查页号是否越界
5 由段表中的页表地址、页号得到查询页表, 找到相应页表项 (第二次: 查页表)
6 由页面存放的内存块号、页内偏移量得到最终的物理地址
8 访问目标单元 (第三次访存)

内存管理

内存空间的扩充（上）

1. 覆盖技术
   - 一个固定区
     - 存放最活跃的程序段
     - 固定区中的程序段在运行过程中不会调入调出
   - 若干覆盖区 —— 不可能被同时访问的程序段可共享一个覆盖区
   - 必须由程序员声明覆盖结构, 操作系统完成自动覆盖
   - 缺点: 对用户不透明, 增加了用户编程负担
   - 在同一个程序或进程中
2. 交换技术
   - 内存紧张时, 换出某些进程以腾出内存空间, 再换入某些进程
   - 磁盘分为文件去和对换去, 换出的进程放在对换区
   - 磁盘分为文件去和对换去, 换出的进程放在对换区

地址转换

- 操作系统负责逻辑地址到物理地址的转换
- 三种方式
  - 绝对装入
  - 静态重新定位
  - 动态重定位

存储保护

- 保证进程在自己的内存空间运行, 不会越界访问
- 两种方式
  - 在CPU中设置一堆上、下限寄存器, 存放进程的上、下限地址
  - 采用重定位寄存器 (基址寄存器) 和界地址寄存器 (限长寄存器) 进行越界检查

- 内存管理
  - 内存空间的扩充（下）
    - 3 虚拟存储技术
      - 基本概念
        - 传统存储管理方式特征
          - **一次性**: 作业数据必须一次全部调入内存
          - 驻留性: 作业数据在整个运行期间都会常驻内存
        - 局部性原理
          - 时间局部性: 现在访问的指令数据在不久后很可能被再次访问
          - 空间局部性: 现在访问的内存单元周围的内存空间很可能在不久后被再次访问高
          - 速缓存技术: 使用频繁的数据放到更高速的存储器中
        - 虚拟存储器
          - 定义
            - 程序不需全部装入即可运行, 运行时根据需要动态调入数据, 若内存不够, 则换出一些数据
          - 特征
            - 多次性: 无需在作业运行时一次性全部装入内存, 允许被分成多次调入内存
            - 对换性: 无需在作业运行时一直常驻内存, 而是允许在作业运行过程中, 将作业换入换出虚
            - 拟性: 从逻辑上扩充了内存的容量, 使用户看到的内存容量远大于实际容量
        - 虚拟内存技术实现
          - 1 请求分页存储管理
          - 2 请求分段存储管理
          - 3 请求段页式存储管理
      - 1 **请求分页存储管理**
        - 1 **页表机制**
          - 在基本分页的基础上增加了几个表项
          - 状态位: 表示页面是否已在内存
          - 访问字段: 记录最近被访问过几次, 或记录上次访问的实践, 供置换算法选择换出页面时参考
          - 修改位: 表示页面调入内存后是否被修改过, 只有修改过的页面才需要在置换时写回外存
          - 外存地址: 页面在外存中存放的位置
        - 2 **缺页中断机构**
          - 找到页表项后检查页面是否已在内存, 若没在内存, 产生缺页中断缺
          - 页中断处理中, 需要将目标页面调入内存, 有必要时还要换出页面
          - 缺页中断属于内中断中的"故障", 可能被系统修复
          - 一条指令在执行过程中可能产生多次缺页中断
        - 3 **地址变换机构**
          - 找到页表项时需检查页面是否在内存中
          - 若页面不在内存中, 需要请求调页
          - 若内存空间不够, 需要换出页面
          - 页面调入内存后, 需要修改相应页表项
      - 2 页面置换算法

算法	算法规则	优缺点
最佳置换算法(OPT)	优先淘汰最长时间内不会再被访问的页面	缺页率最小, 性能最好; 但无法实现
先进先出置换算法(FIFO)	优先淘汰最先进入内存的页面	实现简单; 但性能很差。可能出现Belady异常
最近最久未使用置换算法(LRU	优先淘汰最近最久没访问的页面	性能很好; 但需要硬件支持, 算法开销大
时钟置换算法(CLOCK) [最近未用算法(NRU)]	第一轮淘汰 (访问位=0) 的页面, 并将扫描过的访问位改为0。若第一轮未选中 则进行第二轮扫描。最多进行两轮扫描。	实现简单, 算法开销小; 但未考虑页面是否被修改过
改进型时钟置换算法	用 (访问位,修改位) 形式描述。 第一轮: 淘汰 (0,0) ;第二轮: 淘汰 (0,1) ,并将扫描过的页面访问位都置为0; 第三轮: 淘汰 (0,0) ;第四轮: 淘汰 (0,1) 。 **最多进行四轮扫描。**	算法开销较小, 性能也不错

      - 3 页面分配策略
        - 驻留集
          - 指请求分页存储管理中给进程分配的内存块的集合
        - 工作集
          - 在某段时间间隔里, 进程实际访问页面的集合。驻留集大小一般不能小于工作集大小固
        - 页面分配、置换策略
          - 定分配vs可变分配: 区别在于进程运行期间驻留集大小是否可变
          - 局部置换vs全局置换: 区别在于发生缺页时是否只能从进程自己的页面中选择一个换出
          - 1 固定分配局部置换
            - 进程运行前分配一定数量物理块, 缺页时只能换出进程自己的某一页
          - 2 可变分配全局置换
            - 只要缺页就分配新物理块, 可能来自空闲物理块, 也可能需换出别的进程页面
          - 3 可变分配局部置换
            - 频繁缺页的进程, 多分配一些物理块; 缺页率很低的进程, 回收一些物理块
        - 何时调页
          - 预调页策略
            - 一般用于进程运行前
          - 请求调页策略
            - 进程运行时发现缺页再调页
        - 何处调页
          - 对换区-采用连续存储方式, 速度快; 文件区-采用离散存储方式, 速度慢
          - 对换区足够大: 运行时将数据从文件区复制到对换区, 之后所有页面的调入调出都是在内存与对换区之间进行
          - 对换区不够大: 不会修改的数据每次都从文件区调入; 会修改的数据调出到对换区, 需要时再从对换区调入
          - UNIX方式: 第一次使用的页面都从文件区调入; 调出的页面都写回对换区, 再次使用时从对换区调入
        - 抖动（颠簸）现象
          - 页面频繁换入换出, 主要原因是分配给进程的物理块不够

文件管理

文件定义

一组有意义的信息的集合

文件属性

文件名 (同一目录下不允许有重名文件) 、标识符 (唯一, 对用户无可读性)

类型、位置、大小、保护、时间、日期和用户标识

文件的基本操作

1. 创建文件
   - **"create" 系统调用**
   - 分配外存空间, 创建目录项
2. 删除文件
   - **"delete" 系统调用**
   - 回收外存空间, 删除目录项
3. 打开文件
   - **"open"系统调用**
   - 将目录项中的信息复制到内存中的打开文件表中, 并将打开文件表的索引号 (文件描述符) 返回给用户
   - 打开文件之后, 对文件的操作不再需要每次都查询目录, 可以根据内存中的打开文件表进行操作
   - 每个进程有自己的打开文件表, 系统中只有一张总的打开文件表
   - 并不会把文件数据直接读入内存, 只有在读文件时才数据从外存读入内存进
   - 程打开文件表属性: 读写指针、访问权限、系统表索引号
   - 系统打开文件表属性: 外存地址、打开计数器 (有多少个进程打开了该文件)
4. 关闭文件
   - **"close" 系统调用**
   - 将进程打开文件表中的相应表项删除
   - 系统打开文件表的打开计数器减 1 , 若打开计数器为 0, 则删除系统表的表项
5. 读文件
   - **"read" 系统调用**
   - 根据读指针、读入数据量、内存位置将文件数据从外存读入内存
6. 写文件
   - **"write" 系统调用**
   - 根据写指针、写出数据量、内存位置将文件数据从内存写出外存
   - "读/写文件"用"文件描述符"即可指明文件, 不再需要用到文件名

文件的逻辑结构

- 无结构
  - 由二进制流或字符流组成, 无明显的逻辑结构, 又称"流式文件"。如windows中的".txt"文件
- 有结构
  - 由一组相似的记录组成, 又称"记录式文件", 分为定长记录、可变长记录。每条记录由若干个数据项组成。如数据库表文件。
    - 串结构　记录之间的顺序与关键字无关
    - 顺序结构　记录之间的顺序按关键字顺序排列
    - **链式存储 定长/可变长记录, 都无法实现随机存取, 每次只能从第一个记录开始依次往后查找**
    - **顺序存储**
      - **可变长记录** — **无法实现随机存取。每次只能从第一个记录开始依次往后查找**
      - **定长记录**
        - **可实现随机存取。记录长度为 L则第 i 个记录存放的相对位置是 i*L**
        - **若采用串结构, 无法快速找到某关键字对应的记录**
        - **若采用顺序结构, 可以快速找到某关键字对应的记录 (如折半查找)**
    - **可变长记录的顺序文件无法实现随机存取, 定长记录可以**
    - **定长记录、顺序结构的顺序文件可以快速检索 (根据关键字快速找到记录)**
    - ⚠ 最大缺点: 不方便增加/删除记录
  - 索引文件
    - 建立一张索引表, 每个记录对应一个表项。各记录不用保持顺序, 方便增加/删除记录
    - 索引表本身就是定长记录的顺序文件, 一个索引表项就是一条定长记录, 因此索引文件可支持随机存取若
    - 索引表按关键字顺序排列, 则可支持快速检索
    - 解决了顺序文件不方便增/删记录的问题, 同时让不定长记录的文件实现了随机存取。但索引表可能占用很多空间
  - 索引顺序文件
    - 将记录分组, 每组对应一个索引表项
    - 检索记录时先顺序查索引表, 找到分组, 再顺序查找分组
    - 当记录过多时, 可建立多级索引表

文件管理

文件目录

- 文件控制块 (实现文件目录的关键数据结构)
- 文件目录的实现
  - 对目录的操作: 搜索、创建文件、删除文件、显示文件、修改文件
  - 一个文件对应一个 FCB, 一个 FCB 就是一个目录项, 多个 FCB 组成文件目录
- 目录结构
  1. 两级目录结构 — 不同用户的文件可以重名, 但不能对文件进行分类
  2. 单级目录结构 — 一个系统只有一张目录表, 不允许文件重名
  3. 多级目录结构 (树形目录结构)
     - 系统根据"文件路径"找到目标文件
     - 不同目录下的文件可以重名, 可以对文件进行分类, 不方便文件共享
     - 从根目录出发的路径是"绝对路径" ("1照片1201 5-081自拍.jpg")
     - 从"当前目录"出发的路径是"相对路径"(".1201 5-081自拍.jpg")
  4. 无环图目录结构
     - 为共享结点设置一个共享计数器, 计数器为 0 时才真正删除该结点
     - 在树形目录结构的基础上, 增加一些指向同一节点的有向边, 使整个目录成为一个有向无环图

文件共享

1. 基于索引节点的共享方式 (硬链接)
   - 各个用户的目录项指向同一个索引结点
   - 索引结点中需要有链接计数 count
   - 只有 count == 0 时才能真正删除文件数据和索引结点, 否则会导致指针悬空
   - 某用户想删除文件时, 只是删除该用户的目录项, 且 count - -
2. 索引节点（对文件控制块的优化）
   - 除了文件名之外的所有信息都放到索引结点中, 每个文件对应一个索引结点
   - 目录项中只包含文件名、索引结点指针, 因此每个目录项的长度大幅减小
   - 目录项长度减小→每个磁盘块可以存放更多个目录项→检索文件时磁盘I1O的次数减少
3. 基于符号链的共享方式 (软链接)
   - 即使软链接指向的共享文件已被删除, Link 型文件依然存在,
     只是通过 Link 型文件中的路径去查找共享文件会失败 (找不到对应目录项)
   - 在一个 Link 型的文件中记录共享文件的存放路径 (Windows 快捷方式)
   - 由于用软链接的方式访问共享文件时要查询多级目录, 会有多次磁盘I/O,
     因此用软链接访问共享文件的速度要比硬链接更慢
   - 操作系统根据路径一层层查找目录, 最终找到共享文件

文件保护

1. 口令保护
   - 实现开销小，但"口令"一般存放在 FCB 或索引结点中（也就是存放在系统中）， 安全性低
   - 为文件设置一个"口令", 用户想要访问文件时需要提供口令, 由系统验证口令是否正确
2. 访问控制
   - 用一个访问控制表 (ACL) 记录各个用户 (或各组用户) 对文件的访问权限
   - 对文件的访问类型可以分为:读1写1执行1删除等
   - 如果对某个目录进行了访问权限的控制, 那也要对目录下的所有文件进行相同的访问权限控制
   - 实现灵活, 可以实现复杂的文件保护功能
3. 加密保护
   - 安全性高, 但加密/解密需要耗费一定的时间 (如: 异或加密)
   - 用一个"密码"对文件加密, 用户想要访问文件时, 需要提供相同的"密码"才能正确的解密

文件管理

文件的物理结构（文件分配方式）

1 连续分配

- 要求每个文件在磁盘上占有一组连续的块。
- 优点: **支持顺序访问和直接访问（即随机访问）**；连续分配的文件在顺序访问时速度最快
- 缺点: 不方便文件拓展, 会产生磁盘碎片, 存储空间利用率低

2 链接分配

1 隐式链接

- 除文件的最后一个盘块之外, 每个盘块中都存有指向下一个盘块的指针。
  文件目录包括文件第一块的指针和最后一块的指针。
- 优点: 方便文件拓展, 无碎片问题, 外存利用率高
- 缺点: 只支持顺序访问, 不支持随机访问, 查找效率低, 指向下一个盘块的指针也需耗费少量存储空间
- 考试题目中遇到未指明隐式/显式的"链接分配", 默认指的是隐式链接的链接分配

2 显式链接

- 把用于链接文件各物理块的指针显式地存放在一张表中, 即文件分配表 (FAT)
  一个磁盘只会建立一张文件分配表, 开机时文件分配表放入内存, 并常驻内存
- 优点: 文件拓展方便, 无碎片问题, 外存利用率高, 支持随机访问。相比于隐式链接来说, 地址转换时不需要访问磁盘, 因此文件的访问效率更高
- 缺点: 文件分配表需要占用一定的存储空间

3 索引分配

允许文件离散地分配在各个磁盘块中, 系统会为每个文件建立一张索引表, 索引表中记录了文件的各个逻辑块对应的物理块 (索引表的功能类似于内存管理中的页表——建立逻辑页面到物理页之间的映射关系)

1 链接方案

- 如果索引表太大, 一个索引块装不下, 那么可以将多个索引块链接起来存放
- 缺点: 若文件很大, 索引表很长, 需将很多个索引块链接起来。想要找到 i 号索引块, 必须先依次读入 0~ i-1 号索引块, 导致磁盘I/O次数过多, 查找效率低下

2 多层索引

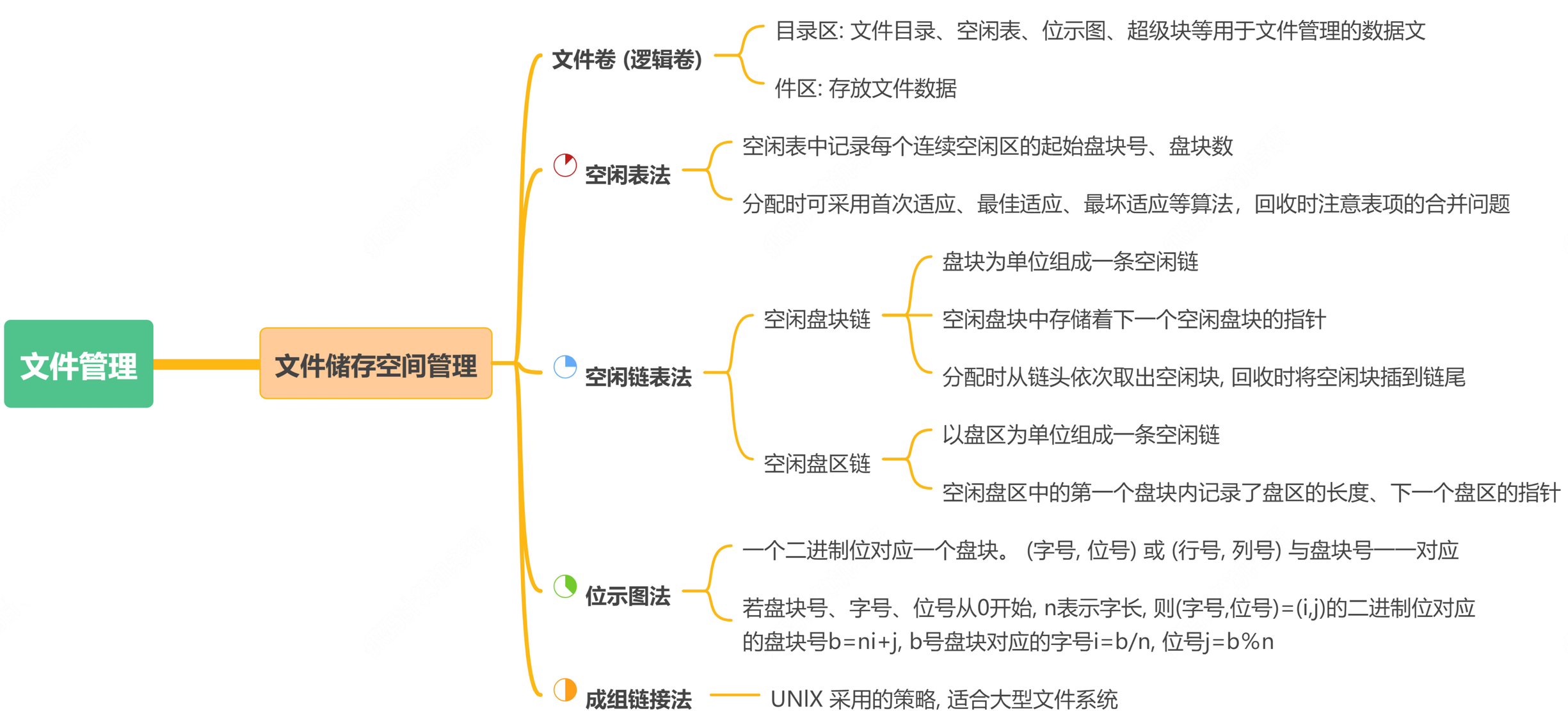
- 建立多层索引 (原理类似于多级页表) 。使第一层索引块指向第二层的索引块。
  还可根据文件大小的要求再建立第三层、第四层索引块
- 采用 K 层索引结构, 且顶级索引表未调入内存, 则访问一个数据块需要 K+1 次读磁盘操作缺点: 即使是小文件, 访问一个数据块依然需要 K+1 次读磁盘。

3 混合索引

- 多种索引分配方式的结合, 文件的顶级索引表中, 既包含直接地址索引 (直接指向数据块) , 又包含一级间接索引 (指向单层索引表) , 还包含两级间接索引 (指向两层索引表)
- 优点: 对于小文件来说, 访问一个数据块所需的读磁盘次数更少

4 重要考点

1. 根据多层索引、混合索引计算出文件的最大长度 (Key: 各级索引表最大不能超过一个块)
2. 分析访问某个数据块所需要的读磁盘次数 (Key: FCB 中会存有指向顶级索引块的指针, 因此可以根据 FCB 读入顶级索引块。每次读入下一级的索引块都需要一次读磁盘操作, 注意题目条件顶级索引块是否已调入内存


文件管理
文件储存空间管理
文件卷 (逻辑卷)
目录区: 文件目录、空闲表、位示图、超级块等用于文件管理的数据文
件区: 存放文件数据
空闲表法
空闲表中记录每个连续空闲区的起始盘块号、盘块数
分配时可采用首次适应、最佳适应、最坏适应等算法，回收时注意表项的合并问题
空闲链表法
空闲盘块链
盘块为单位组成一条空闲链
空闲盘块中存储着下一个空闲盘块的指针
分配时从链头依次取出空闲块, 回收时将空闲块插到链尾
空闲盘区链
以盘区为单位组成一条空闲链
空闲盘区中的第一个盘块内记录了盘区的长度、下一个盘区的指针
位示图法
一个二进制位对应一个盘块。 (字号, 位号) 或 (行号, 列号) 与盘块号一一对应
若盘块号、字号、位号从0开始, n表示字长, 则(字号,位号)=(i,j)的二进制位对应的盘块号b=ni+j, b号盘块对应的字号i=b/n, 位号j=b%n
成组链接法
UNIX 采用的策略, 适合大型文件系统

文件管理

文件系统的层次结构

- **用户接口** — 文件系统需要向上层的用户提供一些简单易用的功能接口。这层就是用于处理用户发出的系统调用请求 (Read、Write、Open、Close 等系统调用)
- **文件目录系统** — 用户是通过文件路径来访问文件的, 因此这一层需要根据用户给出的文件路径找到相应的FCB或索引结点。所有和目录、目录项相关的管理工作都在本层完成, 如: 管理活跃的文件目录表、管理打开文件表等。
- **存取控制模块** — 为了保证文件数据的安全, 还需要验证用户是否有访问权限。这一层主要完成了文件保护相关功能。
- **逻辑文件系统与文件信息缓冲区** — 用户指明想要访问文件记录号, 这一层需要将记录号转换为对应的逻辑地址
- **物理文件系统**
  - 这一层需要把上一层提供的文件逻辑地址转换为实际的物理地址
  - 辅助分配模块 负责文件存储空间的管理, 即分配和回收存储空间
- **设备管理模块** — 直接与硬件交互, 负责和硬件直接相关的一些管理工作。如: 分配设备、分配设备缓冲区、磁盘调度、启动设备、释放设备等
- **设备**

磁盘组织与管理

磁盘的结构

- 磁盘、磁道、扇区
  - 磁盘由表面涂有磁性物质的圆形盘片组成
  - 每个盘片被划分为一个个磁道, 每个磁道又划分为一个个扇区
- 磁盘中读/写数据 — 磁头移动到目标位置, 盘片旋转, 对应扇区划过磁道完成读/写
- 盘面、柱面
  - 磁盘由多个盘片"摞"起来, 每个盘片有两个盘面
  - 所有盘面中相对位置相同的磁道组成柱面
- 磁盘的物理地址 — (柱面号, 盘面号, 扇区号)
- 磁盘的分类
  - 根据磁头是否可移动
    - 固定头磁盘 (每个磁道有一个磁头)
    - 移动头磁盘 (每个盘面只有一个磁头)
  - 根据盘片是否可更换
    - 固定盘磁盘
    - 可换盘磁盘

磁盘速度算法

	思想	评价
先来先服务（FCFS）	按访问请求到达的先后顺序进行处理	优点: 公平; 缺点: 请求访问的磁道分散时寻道时间长, 性能差
最短寻找时间（SSTF）	每次都优先响应距离磁头最近的磁道访问请求, 贪心算法的思想, 能保证眼前最优, 但无法保证总的寻道时间最短	优点: 平均寻道时间短; 缺点: 可能导致饥饿
扫描算法（SCAN）	只有磁头移动到最边缘的磁道时才可以改变磁头移动方向	优点: 平均寻道时间较短, 无饥饿现象; 缺点: 对各个位置磁道的响应频率不平均
循环扫描算法（C—SCAN）	只有磁头朝某个方向移动时才会响应请求, 移动到边缘后立即让磁头返回起点,返回途中不响应任何请求	优点: 比起SCAN算法, 对各个位置磁道的响应频率平均; 缺点: 平均寻道时间更长
LOOK算法	SCAN 算法的改进, 只要在磁头移动方向上不再有请求, 就立即改变磁头方向	\
C-LOOK算法	C-SCAN 算法改进, 只要在磁头移动方向上不再有请求, 就立即让磁头返回	\

一次磁盘读/写操作

- **寻道时间** — 启动磁臂、移动磁头所花的时间
- **延迟时间** — 将目标扇区转到磁头下面所花的时间
- **传输时间** — 读/写数据花费的时间

减少磁盘延迟时间

- 交替编号
  - 具体做法: 让编号相邻的扇区在物理上不相邻
  - 原理: 读取完一个扇区后需要一段时间处理才可以继续读入下一个扇区
- 错位命名
  - 具体做法: 让相邻盘面的扇区编号"错位"
  - 原理: 与"交替编号"的原理相同, "错位命名法"可降低延迟时间
- 磁盘地址 — 采用 (柱面号, 盘面号, 扇区号) 结构, 相邻盘面的相同磁道不用 (盘面号, 柱面号, 扇区号) 结构, 本盘面的相邻磁道原因: 在读取地址连续的磁盘块时, 前者更不需要移动磁头

磁盘的管理

- 磁盘的初始化
  1. 低级格式化/物理格式化: 划分扇区
  2. 磁盘分区: 每个分区由若干柱面组成 (即为C 盘/D 盘/E 盘)
  3. 逻辑格式化: 建立文件系统 (建立根目录文件、建立用于存储空间管理的数据结构)
- 引导快
  - 计算机启动时需要运行初始化程序 (自举程序) 来完成初始化
  - ROM 中存放很小的"自举装入程序
  - 完整的自举程序存放在初始块 (引导块) 中
- 坏块的管理 — 简单的磁盘: 逻辑格式化时在FAT表上将坏块标记出来, 坏块对操作系统不透明复杂的磁盘: 磁盘控制器维护一个坏块链, 低级格式化时对坏块链进行初始化并管理备用扇区, 坏块对操作系统透明

- **I/O管理**
  - I/O管理概述
    - I/O口设备
      - 概念
        - 将数据Input/Output (输入/输出)计算机的外部设备
      - 分类
        - 按使用特性
          - 人机交互类外部设备、存储设备、网络通信设备
        - 按传输速率
          - 低速设备　如键盘、鼠标
          - 中速设备　如行式打印机、激光打印机
          - 高速设备　如磁带机、磁盘机、光盘机
        - 按信息交换的单位
          - 块设备: 传输快, 可寻址, 随机读写任一块
          - 字符设备: 传输慢, 不可寻址, 常采用中断驱动方式
    - I/O控制器
      - 主要功能
        - 控制寄存器: 接受和识别 CPU 发出的命令
        - 状态寄存器: 向 CPU 报告设备的状态
        - 数据寄存器: 数据交换, 暂存输入/输出的数据
        - I/O 逻辑: 地址识别
      - 组成
        - CPU 与控制器之间的接口: 实现控制器与 CPU 之间的通信
        - I/O 逻辑: 负责识别CPU发出的命令, 并向设备发出命令
        - 控制器与设备之间的接口: 实现控制器与设备之间的通信
      - 寄存器编址方式
        - 1 内存映射 I/O
          - 控制器中的寄存器与内存统一编址
          - 可以采用对内存操作的指令来对控制器进行操作
        - 2 寄存器独立编制
          - 控制器中的寄存器独立编址
          - 需要设置专门的指令来操作控制器
    - I/O控制方式

	完成一次读/写的过程	CPU干预频率	每次I/O的数据传输单位	数据流向
程序直接控制方式	CPU 发出 I/O 命令后需要不断轮询	极高	字	设备→CPU→内存　内存→CPU→设备
中断驱动方式	CPU 发出 I/O 命令后可以做其他事, 本次/O完成后设备控制器发出中断信号	高	字	设备→CPU→内存　内存→CPU→设备
DMA方式	CPU 发出 I/O 命令后可以做其他事, 本次/O完成后 DMA 控制器发出中断信号	中	块	设备→内存　内存→设备
通道空置方式	CPU 发出 I/O 命令后可以做其他事。 通道会执行通道 程序以完成/O,完成后通道向 CPU 发出中断信号	低	一组块	设备→内存　内存→设备

    - I/O软件层次
      - 1 用户层I/O 软件
        - 实现与用户交互的接口, 向上提供方便易用的库函数; **假脱机技术**
      - 2 设备独立性软件
        - 向上层提供统一的调用接口 (如read/wr ite系统调用) ; 设备的保护; 差错处理;
          设备的分配与回收; 数据缓冲区管理; 建立逻辑设备名到物理设备名的的映射关系,
          根据设备类型选择调用相应的驱动程序
      - 3 设备驱动程序
        - 设置设备寄存器、检查设备状态
      - 4 中断处理程序
        - 进行中断处理
      - 5 硬件设备
        - 执行I/O 操作, 由机械部分、电子部分 (I/O 控制器) 组成

- I/O管理
  - I/O核心子系统
    - 假脱机技术
      - 脱机技术
        - 外围控制机+更高速的设备磁带
        - 作用: 缓解设备与 CPU 的速度矛盾, 实现预输入、缓输出又
      - 假脱机技术
        - **又叫 SPOOLing 技术, 用软件的方式模拟脱机技术**
        - 系统组成
          1. 输入井和输出井一模拟脱机输入/输出时的磁带
          2. 输入进程和输出进程一模拟脱机输入/输出时的外围控制机
          3. 输入缓冲区和输出缓冲区一内存中的缓冲区, 输入输出时的"中转站"
      - 共享打印机 — 用 SPOOLing 技术将独占式的打印机"虚拟"成共享打印机
    - 缓存区管理
      - 缓冲区
        - 利用硬件作为缓冲区的成本较高, 容量较小, 如TLB。一般利用内存作为缓冲区
        - 缓解 CPU 与设备的速度矛盾、减少对 CPU 的中断频率及放款对CPU中断响应时间的限制、解决数据粒度不匹配的问题、提高 CPU 与 I/O 设备之间的并行性
      1. **单缓冲**
         - 设备输入 (T) 、缓冲区传送 (M) 、工作区处理(C)
         - 分析问题的初始状态: 工作区满, 缓冲区空
         - 处理一块数据平均耗时 Max (C, T) +M
      2. **双缓冲** — 分析问题的初始状态: 工作区空, 一个缓冲区满, 另一个缓冲区空处理一块数据平均耗时 Max (T, C+M)
      3. **循环缓冲** — 多个缓冲区链接成循环队列, in 指针指向下一个可以冲入数据的空缓冲区, out 指针指向下一个可以取出数据满缓冲区
      4. **缓冲池**
         - 缓冲队列
           - 空缓冲队列
           - 输入队列
           - 输出队列
         - 工作缓冲区
           - 用于收容输入数据的工作缓冲区 (hin)
           - 用于提取输入数据的工作缓冲区 (sin)
           - 用于收容输出数据的工作缓冲区 (hout)
           - 用于提取输出数据的工作缓冲区 (sout)
    - 设备的分配与回收
      - 应考虑的因素
        - **固有属性** — **独占设备、共享设备、虚拟设备 (SPOOLing)**
        - **分配算法** — **先来先服务、优先级高者优先、短任务优先等**
        - 安全性
          - 安全分配方式 — 为进程分配一个设备后就将进程阻塞, 本次I/O完成后才将进程唤醒
          - 不安全分配方式 — 系统为进程分配I/O设备, 进程可继续执行
      - 分配策略
        - 静态分配: 进程运行前为其分配全部所需资源, 运行结束后归还资源
        - 动态分配: 进程运行过程中动态申请设备资源
      - 设备分配管理中的数据结构
        - **设备控制表 (DCT)**
          - 每个设备对应一张 DCT
          - 关键字段: 类型/标识符/状态/指向COCT的指针/等待队列指针
        - **控制器控制表 (COCT)**
          - 每个控制器对应一张 COCT
          - 关键字段: 状态/指向CHCT的指针/等待队列指针
        - **通道控制表 (CHCT)**
          - 每个控制器对应一张 CHCT
          - 关键字段: 控制器表首址/状态等待队列指针
        - **系统设备表 (SDT)**
          - 每个设备对应一个表目, 记录整个系统中所有设备的情况
          - 关键字段: 设备类型/标识符/DCT/驱动程序入口
      - 设备分配的步骤
        1. 根据进程请求的物理设备名 (设备标识符) 查找 SDT
        2. 根据 SDT 找到 DCT 并分配设备
        3. 根据 DCT 找到 COCT 并分配控制器
        4. 根据 COCT 找到 CHCT 并分配通道
        - 只有设备、控制器、通道三者都分配成功时, 这次设备分配才算成功, 之后便可启动/O 设备进行数据传送
        - 缺点:
          用户编程时必须使用"物理设备名", 若换了一个物理设备, 则程序无法运行; 若进程请求的物理设备正在忙碌, 则即使系统中还有同类型的设备, 进程也必须阻塞等待用户编程时使用逻辑设备名申请设备, 操作系统负责实现从逻辑设备名到物理设备名的映射 (通过 LUT)
      - 设备分配步骤的改进 — 逻辑设备表的设置问题
        - 整个系统只有一张 LUT: 各用户所用的逻辑设备名不允许重复, 适用于单用户操作系统
        - 每个用户一张 LUT: 各个用户的逻辑设备名可重复, 适用于多用户操作系统